



MÓDULO 6 · T16

Calidad y Deploy: Testing con Jest

 *Tests unitarios y de componente*

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

¿Qué aprenderás hoy?

En esta sesión construirás una base sólida sobre cómo verificar la corrección de tus componentes, funciones y hooks con Jest y React Native Testing Library. Al terminar, serás capaz de escribir tests confiables en proyectos reales.

1

Configurar Jest

Instalar y configurar Jest con React Native Testing Library en un proyecto Expo.

2

Tests de componente

Escribir tests usando `render` y `screen` para verificar el comportamiento visual.

3

Tests de funciones puras

Testear mappers y utils sin mocks ni render – los tests más rápidos y directos.

4

Tests de hooks

Usar `renderHook` y `act` para testear la lógica de hooks de forma aislada.

Instalación

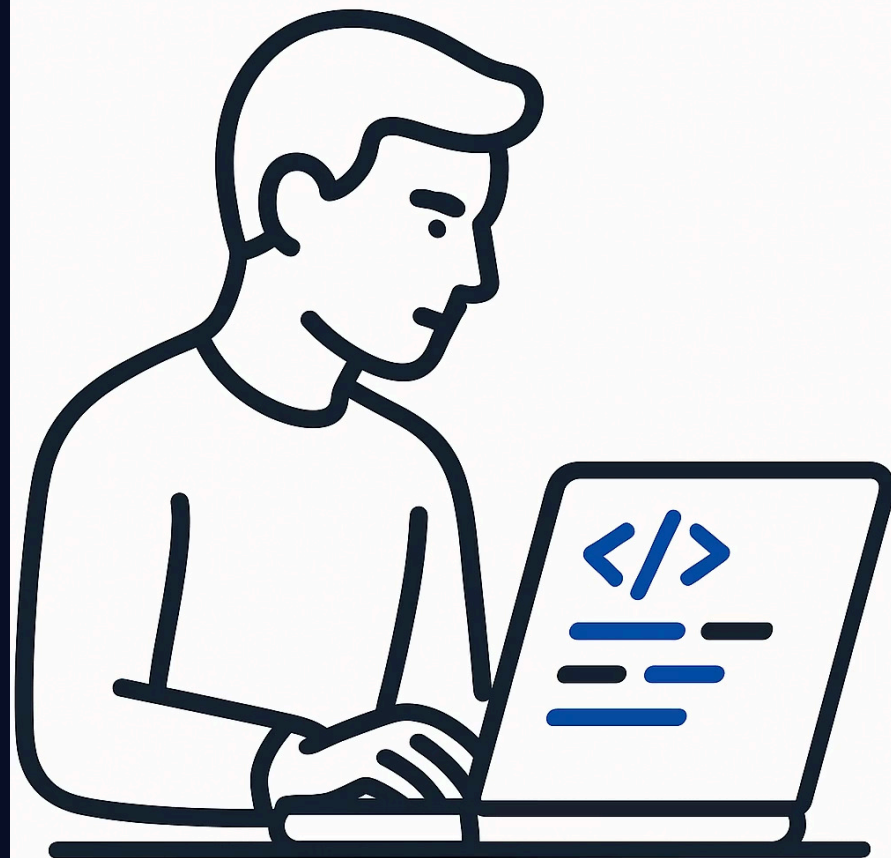
Expo ya incluye Jest preconfigurado. Solo es necesario añadir la Testing Library y registrar su configuración en `jest.config.js` o `package.json`:

```
npx expo install @testing-library/react-native @testing-library/jest-native
```

Configura el preset y el setup en `package.json`:

```
"jest": {  
  "preset": "jest-expo",  
  "setupFilesAfterFramework": [  
    "@testing-library/jest-native/extend-expect"  
  ]  
}
```

❗ El preset `jest-expo` ya transforma JSX y alias de módulo – no necesitas configuración adicional para la mayoría de proyectos.



Test de componente

¿Por qué testear componentes?

Un test de componente verifica que el árbol de UI renderizado contiene exactamente lo que el usuario verá. Detecta regresiones visuales sin necesidad de ejecutar la app completa.

Con `screen.getByText` buscas por texto visible – igual que lo haría un usuario real buscando en pantalla.

- ✓ Si el componente renderiza el texto correcto, el test pasa – sin inspeccionar implementación interna.

Ejemplo con `PokemonTypeBadge`

```
import { render, screen } from '@testing-library/react-native';
import PokemonTypeBadge from
'../src/components/PokemonTypeBadge';

describe('PokemonTypeBadge', () => {
  it('muestra el nombre del tipo', () => {
    render(<PokemonTypeBadge tipo="fuego" />);
    expect(screen.getByText('fuego')).toBeTruthy();
  });
});
```

Verificar estilos

Además del texto visible, puedes verificar que el componente aplica los estilos correctos inspeccionando las props del elemento renderizado. El helper `expect.objectContaining` permite hacer un match parcial sin listar todos los estilos.

```
it('aplica el color de fondo correcto', () => {
  const { getByText } = render(
    <PokemonTypeBadge tipo="planta" />
  );
  const label = getByText('planta');
  const badge = label.parent?.parent; // Text → View wrapper → View badge

  expect(badge?.props.style).toEqual(
    expect.arrayContaining([
      expect.objectContaining({
        backgroundColor: tipoColor['planta']
      }),
    ])
  );
});
```

- ❶ `expect.arrayContaining` + `expect.objectContaining` permiten verificar que un estilo concreto está presente dentro de un array de estilos, sin importar los demás estilos del componente.

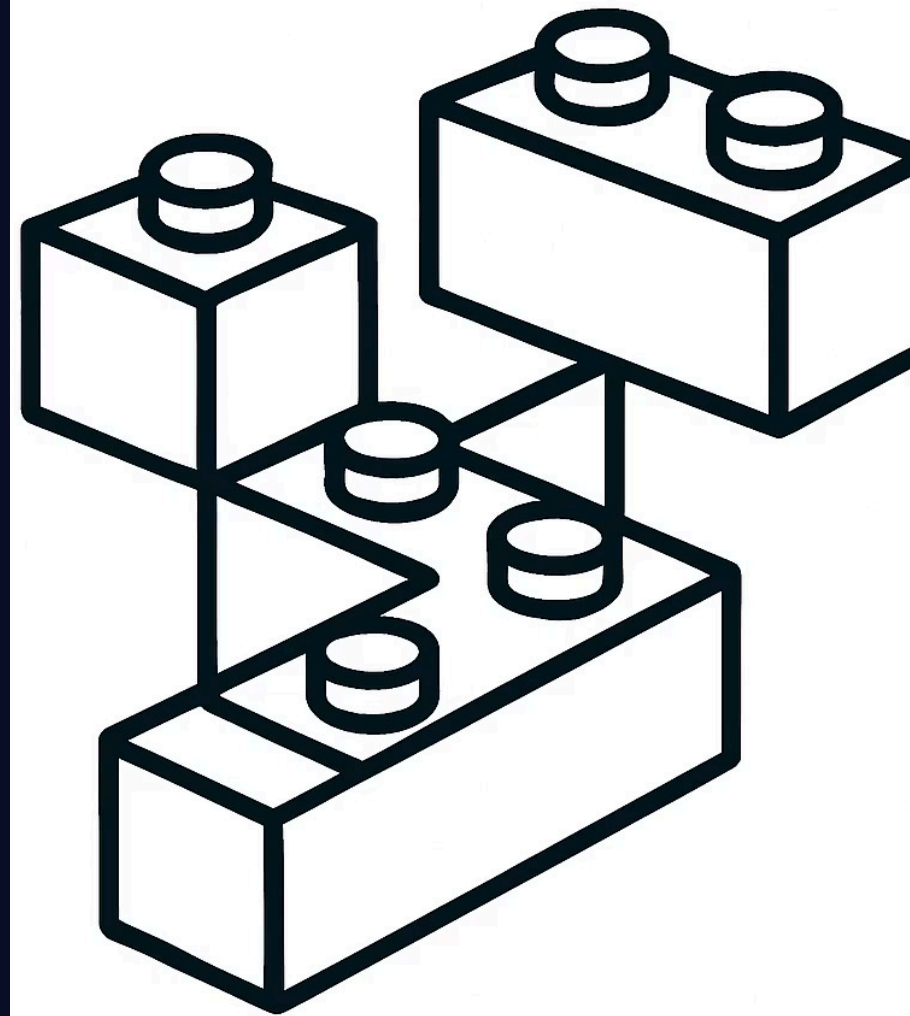
Test de función pura (mapper)

Las funciones puras como mappers y utils son las más fáciles de testear: reciben una entrada, devuelven una salida, y **no tienen efectos secundarios**. No necesitan render ni mocks – solo datos de entrada y aserciones.

```
import { mapToPokemon } from '../src/utils/mapToPokemon';
```

```
describe('mapToPokemon', () => {  
  it('mapea id y nombre', () => {  
    const p = mapToPokemon(  
      mockPokemonResponse,  
      mockSpeciesResponse  
    );  
    expect(p.id).toBe(6);  
    expect(p.nombre).toBe('charizard');  
  });  
  
  it('mapea los tipos al español', () => {  
    const p = mapToPokemon(  
      mockPokemonResponse,  
      mockSpeciesResponse  
    );  
    expect(p.tipos).toContain('fuego');  
    expect(p.tipos).toContain('volador');  
  });  
});
```

- ✓ Funciones puras = sin mocks, sin render – los tests más rápidos y fáciles de mantener.



Fixtures — datos de prueba

En lugar de llamar a la API real en los tests, crea objetos que imiten la respuesta. Estos **fixtures** hacen los tests deterministas, rápidos y sin dependencias de red. Solo incluye los campos que tu mapper realmente utiliza.

```
// En lugar de llamar a la API real en los tests,  
// crea un objeto que imite la respuesta:  
const mockPokemonResponse = {  
  id: 6,  
  name: 'charizard',  
  order: 7,  
  types: [  
    { type: { name: 'fire' } },  
    { type: { name: 'flying' } }  
  ],  
  stats: [  
    { base_stat: 78, stat: { name: 'hp' } },  
    // ...  
  ],  
  // Solo los campos que tu mapper utiliza  
};
```

📄 **Buena práctica:** coloca tus fixtures en una carpeta `__fixtures__` o `__mocks__` y reutilízalos entre suites de test para evitar duplicar datos de prueba.

Test de hook con renderHook

Inicialización

```
import { renderHook, act }
  from '@testing-library/react-native';
import { useBattleState }
  from '../src/hooks/useBattleState';

it('inicializa el HP desde stats.hp', () => {
  const { result } = renderHook(() =>
    useBattleState([pokemon1], [pokemon2])
  );
  expect(
    result.current.topHP[0]?.current
  ).toBe(200);
});
```

Actualizaciones asíncronas

```
it('el ataque reduce el HP', async () => {
  await act(async () => {
    await result.current.attack(playCry);
  });

  expect(
    result.current.topHP[0]?.current
  ).toBeLessThan(200);
});
```

Usa `act` para envolver cualquier acción que provoque actualizaciones de estado – de lo contrario Jest advertirá de actualizaciones sin envolver.

 `renderHook` monta el hook en un componente anónimo de test y expone su valor de retorno a través de `result.current`. Úsalo siempre que quieras testear lógica de hook sin asociarla a un componente real.

Tour starter/MyComponent.test.tsx

Este es el fichero que encontrarás en la carpeta `starter/`. Contiene la estructura básica de una suite de tests con dos casos marcados como `TODO`. Tu misión es reemplazar los placeholders con aserciones reales sobre el componente que construiste en T03.

```
describe('MyComponent', () => {  
  it('renderiza sin errores', () => {  
    // TODO: render(<MyComponent prop="valor" />);  
    expect(true).toBe(true); // placeholder  
  });  
  
  it('muestra el texto correcto', () => {  
    // TODO  
  });  
});
```

⚠ No modifiques la estructura del `describe` ni los nombres de los casos de test – céntrate en implementar el cuerpo de cada `it`.

Los 3 tipos de test

Cada tipo de código en tu app tiene su estrategia de test óptima. Aquí tienes los tres patrones de referencia para componentes, funciones puras y hooks.

Componente

```
render(  
  <Badge tipo="fuego" />  
);  
screen.getByText('fuego');  
fireEvent.press(btn);
```

Función pura

```
const p = mapToPokemon(  
  mock,  
  species  
);  
expect(p.nombre).toBe('x');  
// sin render, sin mocks
```

Hook

```
const { result } = renderHook(  
  () => useMyHook();  
  act(() =>  
    result.current.doSomething();  
  expect(result.current.value).toBe(x);
```

- ✓ Dominar estos tres patrones cubre prácticamente el 100% de los escenarios de test que encontrarás en una app React Native real.

Ejercicio T16

Pon en práctica todo lo aprendido. Sigue los pasos en orden – cada uno construye sobre el anterior. El bonus es opcional pero muy recomendable para afianzar el patrón de `renderHook`.

1 Ejecuta la suite existente

Lanza `npx jest` en PokeApp y verifica que pasan los 3 suites incluidos.

2 Estudia la solución de referencia

Revisa `solucion/__tests__/PokemonTypeBadge.test.tsx` y comprende cada aserción.

3 Crea un test de componente

Escribe un test para el componente que construiste en T03.

4 Escribe un test de mapper

Crea una suite de tests para tu mapper del T15 usando fixtures.

5 Bonus: `renderHook` para el hook del T09

Usa `renderHook` y `act` para testear la lógica de tu hook personalizado.

Resumen de la sesión

Estas son las herramientas clave que has trabajado hoy. Dominarlas te permitirá escribir tests fiables para cualquier capa de tu app React Native.

Herramienta	Uso	Para qué sirve
<code>render</code>	<code>render(<C />)</code>	Monta el componente en el árbol de test
<code>screen.getByText</code>	<code>screen.getByText('x')</code>	Busca por texto visible
<code>fireEvent.press</code>	<code>fireEvent.press(btn)</code>	Simula un toque
<code>expect.objectContaining</code>	<code>expect.objectContaining({...})</code>	Match parcial de objeto
Fixtures	Objetos JS estáticos	Datos de prueba sin llamadas reales
<code>renderHook</code>	<code>renderHook(() => useHook())</code>	Testa hooks directamente
<code>act</code>	<code>await act(async () => {...})</code>	Envuelve actualizaciones asíncronas

✅ ¡Enhorabuena! Ahora tienes las herramientas para garantizar la calidad de componentes, funciones y hooks en cualquier proyecto React Native con Jest. 🚀